

TECHNICAL STANDARD OPERATING PROCEDURE

PostgreSQL Index Administration Toolkit

Discovery, Usage, Size, Bloat, Fragmentation and REINDEX Decision Support

Field	Value
Technology / Platform	PostgreSQL
Target Version	PostgreSQL 18 (validated against PostgreSQL 18 documentation)
Document Type	Technical SOP / DBA Operations Runbook
Document Version	1.0
Prepared Date	27 August 2026
Classification	Internal Technical Reference

Document Control

Field	Details
Document Title	PostgreSQL Index Administration Toolkit
Purpose	Provide a safe, reusable toolkit for discovering, measuring, interpreting and maintaining PostgreSQL indexes.
Scope	User indexes in a PostgreSQL 18 database, with emphasis on B-tree analysis and operational REINDEX decisions.
Technology / Platform	PostgreSQL 18
Target Version	18.x
Document Version	1.0
Primary Documentation Source	Official PostgreSQL 18 documentation
Intended Audience	PostgreSQL DBAs, SREs, platform engineers and performance engineers
Document Owner	Database Engineering / DBA Team
Classification	Internal Technical Reference
Review Trigger	PostgreSQL major-version upgrade, extension behavior change, or operational policy change

Revision History

Version	Date	Description
1.0	27 August 2026	Initial release

Source and Accuracy Statement

This SOP was prepared and cross-checked against the official PostgreSQL documentation cited in the Official References section.

Scope:

index discovery, usage analysis, unused-index detection, size analysis, B-tree identification, pgstatindex() integration, dynamic SQL, reusable functions, bloat estimation, REINDEX recommendations, largest-index reporting, fragmentation, density and interpretation guidance.

Production validation is still required against the exact PostgreSQL minor release, operating system, infrastructure, extensions, security model, workload, maintenance windows and organizational change controls. Index statistics are observational data, not an automatic instruction to drop or rebuild an index.

TABLE OF CONTENTS / INDEX

1. Introduction	5
2. Purpose, Scope and Objectives.....	5
3. Architecture and Data Flow	5
4. Prerequisites and Safety	6
5. Install and Validate pgstattuple	6
6. Index Discovery	6
7. Usage Analysis	7
8. Unused Index Candidate Detection	7
9. Size Analysis and Largest Indexes	8
10. B-tree Identification.....	8
11. pgstatindex() Integration	8
12. Reusable B-tree Health Function	9
13. Database-Wide B-tree Fragmentation and Density Report.....	10
14. Bloat Estimation: What This Toolkit Can Safely Claim	10
15. Dynamic SQL Generation	11
16. REINDEX Recommendation Framework	11
17. REINDEX Progress Monitoring	12
18. Complete Index Administration Dashboard Query	12
19. Interpretation Guidelines	12
20. Real-World Production Scenarios	13
21. Troubleshooting.....	13
22. Security and Least Privilege	13
23. Performance and Operational Impact	14
24. Production Readiness Checklist	14
25. Post-Implementation Validation.....	14
26. Operational Cheat Sheet.....	15
27. Key Takeaways	15
28. Official References	15
Appendix A - Complete Candidate Review Script	15
Appendix B - DBA Decision Matrix.....	17

1. Introduction

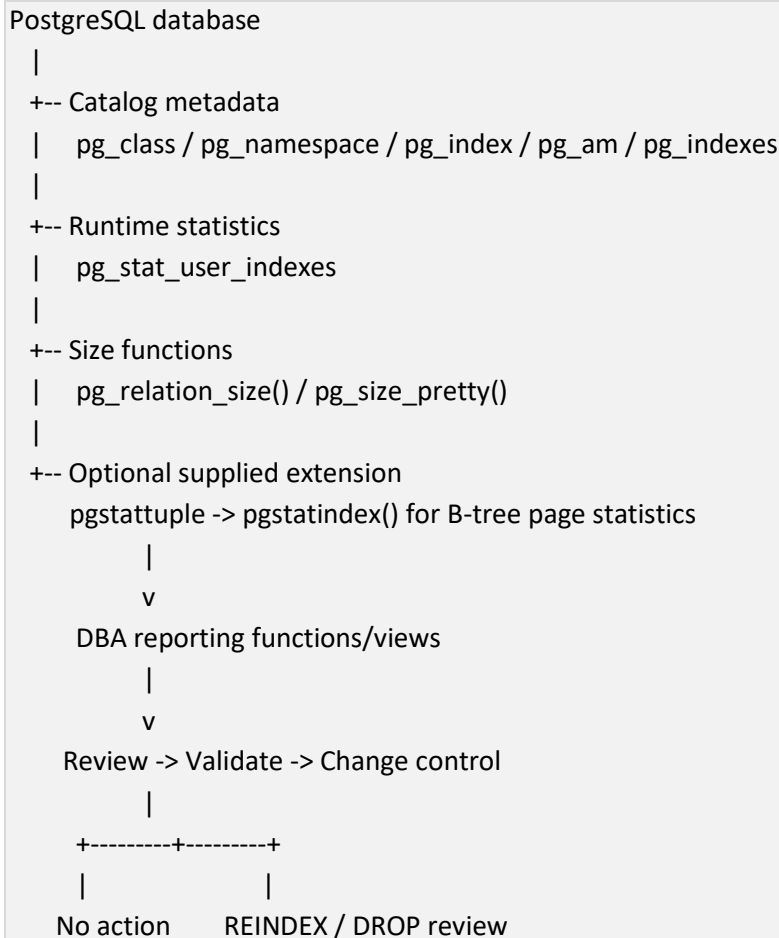
Indexes can make row retrieval much faster, but every index also consumes storage and adds write-maintenance overhead. PostgreSQL therefore recommends using indexes sensibly and examining actual index usage before deciding whether an index is useful.

This toolkit combines PostgreSQL catalog views, cumulative statistics views, size functions and the supplied pgstattuple extension to create a practical DBA workflow. It deliberately separates observation from action: reports identify candidates; a DBA validates workload context before DROP INDEX or REINDEX.

2. Purpose, Scope and Objectives

- Discover indexes, their definitions, access methods, validity, uniqueness and table ownership.
- Measure usage using pg_stat_user_indexes, including idx_scan and last_idx_scan.
- Identify possible unused indexes without automatically declaring them safe to drop.
- Measure index size and rank the largest indexes.
- Identify B-tree indexes before calling pgstatindex(), because pgstatindex() is for B-tree indexes.
- Report B-tree leaf density and leaf fragmentation using pgstattuple/pgstatindex().
- Generate review SQL and REINDEX commands dynamically without executing destructive actions.
- Provide conservative interpretation rules for bloat and REINDEX decisions.

3. Architecture and Data Flow



The toolkit is read-oriented. The only maintenance commands it generates are recommendations. The DBA remains responsible for workload validation, lock impact, backup/recovery posture and change approval.

4. Prerequisites and Safety

Requirement	Guidance
PostgreSQL	18.x target for this SOP.
Privileges	Catalog/statistics access is generally available, but pgstattuple functions are restricted by default; use a superuser or appropriately authorized role such as a member of pg_stat_scan_tables.
Extension	CREATE EXTENSION pgstattuple; must be installed in each database where pgstatindex() is required.
Statistics history	Do not interpret idx_scan=0 without knowing whether statistics were reset or the observation window is representative.
Change control	Never automatically DROP or REINDEX from an unused/bloat report.
Production locking	Standard REINDEX blocks table writes; REINDEX CONCURRENTLY reduces write blocking but costs more work and time.

4.1 Pre-Implementation Checklist

- Confirm the target database and PostgreSQL version.
- Record the observation window and whether statistics were recently reset.
- Confirm the pgstattuple extension is approved before installation.
- Run reports first in a non-production environment when possible.
- Exclude or separately review primary-key, unique and constraint-supporting indexes.
- Use EXPLAIN (ANALYZE, BUFFERS) on representative queries before removing an index.
- Plan REINDEX work around I/O, CPU, WAL and replica impact.

5. Install and Validate pgstattuple

pgstattuple is a supplied PostgreSQL extension. pgstatindex(regclass) reports B-tree structure information including index_size, leaf_pages, empty_pages, deleted_pages, avg_leaf_density and leaf_fragmentation.

```
CREATE EXTENSION IF NOT EXISTS pgstattuple;

SELECT extname, extversion
FROM pg_extension
WHERE extname = 'pgstattuple';
```

Expected result: one row for pgstattuple. If a non-superuser will run page-level functions, grant only the required access according to local security policy; membership in pg_stat_scan_tables is usually preferable to broad grants.

6. Index Discovery

pg_indexes is convenient for schema/table/index names and reconstructed index definitions. For deeper administration, join pg_class, pg_namespace, pg_index and pg_am.

```
SELECT
  n.nspname AS schema_name,
  t.relname AS table_name,
  i.relname AS index_name,
  am.amname AS access_method,
  ix.indisunique AS is_unique,
  ix.indisprimary AS is_primary,
```

```

ix.indisvalid AS is_valid,
ix.indisready AS is_ready,
pg_size_pretty(pg_relation_size(i.oid)) AS index_size,
pg_get_indexdef(i.oid) AS index_definition
FROM pg_class AS i
JOIN pg_index AS ix ON ix.indexrelid = i.oid
JOIN pg_class AS t ON t.oid = ix.indrelid
JOIN pg_namespace AS n ON n.oid = i.relnamespace
JOIN pg_am AS am ON am.oid = i.relam
WHERE i.relkind IN ('i','I')
AND n.nspname NOT IN ('pg_catalog', 'information_schema')
ORDER BY n.nspname, t.relname, i.relname;

```

7. Usage Analysis

pg_stat_user_indexes provides one row per user index. idx_scan is the number of index scans initiated; last_idx_scan records the last scan time based on the most recent transaction stop time. idx_tup_read and idx_tup_fetch help describe entries read and live heap rows fetched by simple index scans.

```

SELECT
  schemaname,
  relname AS table_name,
  indexrelname AS index_name,
  idx_scan,
  last_idx_scan,
  idx_tup_read,
  idx_tup_fetch
FROM pg_stat_user_indexes
ORDER BY idx_scan DESC, schemaname, relname, indexrelname;

```

8. Unused Index Candidate Detection

An index with idx_scan = 0 is only a candidate for review. It may support a primary key or uniqueness rule, a rare but critical report, a standby-only workload, or a workload that has not occurred since statistics were reset.

```

SELECT
  s.schemaname,
  s.relname AS table_name,
  s.indexrelname AS index_name,
  pg_size_pretty(pg_relation_size(s.indexrelid)) AS index_size,
  s.idx_scan,
  s.last_idx_scan,
  i.indisprimary,
  i.indisunique
FROM pg_stat_user_indexes AS s
JOIN pg_index AS i ON i.indexrelid = s.indexrelid
WHERE s.idx_scan = 0
ORDER BY pg_relation_size(s.indexrelid) DESC;

```

Recommended review rule: do not auto-drop indexes where indisprimary or indisunique is true. Also inspect pg_constraint, query plans, application release cycles and the statistics reset/observation period.

9. Size Analysis and Largest Indexes

```
SELECT
  schemaname,
  relname AS table_name,
  indexrelname AS index_name,
  pg_relation_size(indexrelid) AS index_bytes,
  pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
ORDER BY pg_relation_size(indexrelid) DESC
LIMIT 25;
```

Large size alone is not a defect. A large, frequently used index may be healthy and valuable. Size becomes actionable when combined with workload, growth, page density, fragmentation, storage pressure and maintenance cost.

10. B-tree Identification

```
SELECT
  n.nspname AS schema_name,
  t.relname AS table_name,
  i.relname AS index_name,
  i.oid::regclass AS index_regclass
FROM pg_class AS i
JOIN pg_index AS ix ON ix.indexrelid = i.oid
JOIN pg_class AS t ON t.oid = ix.indrelid
JOIN pg_namespace AS n ON n.oid = i.relnamespace
JOIN pg_am AS am ON am.oid = i.relam
WHERE i.relkind = 'i'
  AND am.amname = 'btree'
  AND n.nspname NOT IN ('pg_catalog', 'information_schema')
ORDER BY 1,2,3;
```

Partitioned index parents have relkind='I' and do not contain the same physical leaf pages as ordinary index relations. Run pgstatindex() against physical B-tree index relations, normally relkind='i'.

11. pgstatindex() Integration

```
SELECT *
FROM pgstatindex('public.orders_created_at_idx'::regclass);
```

Metric	Meaning
index_size	Total B-tree index size in bytes.
tree_level	Root tree level; larger trees have more levels to traverse.
leaf_pages	Number of leaf pages.
empty_pages	Completely empty pages.
deleted_pages	Deleted pages reported by the B-tree inspection.

avg_leaf_density	Average percentage of leaf-page space occupied.
leaf_fragmentation	Leaf-page fragmentation percentage reported by pgstatindex().

pgstatindex() reads page-level information and its result is accumulated page by page; concurrent activity means it should not be treated as a perfectly instantaneous snapshot.

12. Reusable B-tree Health Function

The following SQL function returns pgstatindex() metrics for one B-tree index. It validates the access method before inspection and uses regclass to avoid unsafe string concatenation.

```
CREATE OR REPLACE FUNCTION dba_btree_index_health(p_index regclass)
RETURNS TABLE (
    index_name text,
    index_size_bytes bigint,
    tree_level integer,
    leaf_pages bigint,
    empty_pages bigint,
    deleted_pages bigint,
    avg_leaf_density double precision,
    leaf_fragmentation double precision
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_am name;
BEGIN
    SELECT am.amname
    INTO v_am
    FROM pg_class c
    JOIN pg_am am ON am.oid = c.relam
    WHERE c.oid = p_index
    AND c.relkind = 'i';

    IF v_am IS DISTINCT FROM 'btree' THEN
        RAISE EXCEPTION '% is not a physical B-tree index', p_index;
    END IF;

    RETURN QUERY
    SELECT
        p_index::text,
        s.index_size,
        s.tree_level,
        s.leaf_pages,
        s.empty_pages,
        s.deleted_pages,
        s.avg_leaf_density,
        s.leaf_fragmentation
```



```

FROM pgstatindex(p_index) AS s;
END;
$$;

SELECT *
FROM dba_btree_index_health('public.orders_created_at_idx'::regclass);

```

13. Database-Wide B-tree Fragmentation and Density Report

Calling `pgstatindex()` across every large index can create meaningful I/O. Run this during a controlled diagnostic window and filter by size or schema.

```

SELECT
  n.nspname AS schema_name,
  t.relname AS table_name,
  i.relname AS index_name,
  pg_size_pretty(pg_relation_size(i.oid)) AS index_size,
  x.avg_leaf_density,
  x.leaf_fragmentation,
  x.empty_pages,
  x.deleted_pages
FROM pg_class i
JOIN pg_index ix ON ix.indexrelid = i.oid
JOIN pg_class t ON t.oid = ix.indrelid
JOIN pg_namespace n ON n.oid = i.relnamespace
JOIN pg_am am ON am.oid = i.relam
CROSS JOIN LATERAL pgstatindex(i.oid::regclass) AS x
WHERE i.relkind = 'i'
  AND am.amname = 'btree'
  AND n.nspname NOT IN ('pg_catalog', 'information_schema')
  AND pg_relation_size(i.oid) >= 100 * 1024 * 1024
ORDER BY x.leaf_fragmentation DESC,
  x.avg_leaf_density ASC;

```

14. Bloat Estimation: What This Toolkit Can Safely Claim

PostgreSQL does not expose a single built-in percentage named 'index bloat'. `pgstatindex()` gives physical B-tree indicators such as leaf density, fragmentation, empty pages and deleted pages. These are useful signals, but they are not a universal exact bloat percentage.

A conservative operational estimate can report potentially reclaimable free space inside leaf pages, but it must be labeled as an estimate. Fillfactor, page splits, workload shape and index design affect what 'healthy' density looks like.

```

SELECT
  x.index_size,
  x.avg_leaf_density,
  x.leaf_fragmentation,
  round((100.0 - x.avg_leaf_density)::numeric, 2) AS leaf_free_pct_estimate
FROM pgstatindex('public.orders_created_at_idx'::regclass) AS x;

```

Interpret leaf_free_pct_estimate as a diagnostic signal, not as guaranteed reclaimable disk space.

15. Dynamic SQL Generation

Generate commands for human review rather than executing them automatically.

```
SELECT format(
  'REINDEX INDEX CONCURRENTLY %I.%I;',
  schemaname,
  indexrelname
) AS proposed_command
FROM pg_stat_user_indexes
WHERE pg_relation_size(indexrelid) >= 1024^3
ORDER BY pg_relation_size(indexrelid) DESC;
```

The size filter above is only an example candidate filter. It is not a PostgreSQL recommendation threshold.

16. REINDEX Recommendation Framework

Observation	Recommended DBA interpretation
Index is corrupted or invalid	REINDEX is a supported recovery/rebuild mechanism; validate the specific failure and dependency context.
B-tree has persistent inefficient space use	Consider REINDEX after confirming the pattern and business impact.
Storage parameter such as fillfactor changed	REINDEX can rebuild the index so the change takes full effect.
Production system must keep writes flowing	Prefer evaluating REINDEX ... CONCURRENTLY; it performs more work and takes longer but avoids locks that block normal writes for the rebuild.
idx_scan = 0 only	Do not REINDEX or DROP solely from this metric.
Large index only	No action solely because it is large.

```
-- Single index, standard rebuild:
REINDEX INDEX public.orders_created_at_idx;

-- Production-oriented alternative when supported:
REINDEX INDEX CONCURRENTLY public.orders_created_at_idx;

-- All indexes on a table:
REINDEX TABLE CONCURRENTLY public.orders;
```

REINDEX CONCURRENTLY cannot run inside a transaction block. It has important caveats, including extra scans/work, waiting for transactions, temporary indexes, and unsupported cases such as exclusion-constraint indexes for concurrent reindexing.

17. REINDEX Progress Monitoring

```
SELECT
  pid,
  datname,
  relid::regclass AS table_name,
  index_relid::regclass AS index_name,
```

```

command,
phase,
lockers_total,
lockers_done,
blocks_total,
blocks_done,
tuples_total,
tuples_done
FROM pg_stat_progress_create_index
ORDER BY pid;
```

PostgreSQL reports REINDEX progress through `pg_stat_progress_create_index`.

18. Complete Index Administration Dashboard Query

```

SELECT
  s.schemaname,
  s.relname AS table_name,
  s.indexrelname AS index_name,
  am.amname AS access_method,
  i.indisprimary AS is_primary,
  i.indisunique AS is_unique,
  i.indisvalid AS is_valid,
  s.idx_scan,
  s.last_idx_scan,
  s.idx_tup_read,
  s.idx_tup_fetch,
  pg_relation_size(s.indexrelid) AS index_bytes,
  pg_size_pretty(pg_relation_size(s.indexrelid)) AS index_size
FROM pg_stat_user_indexes s
JOIN pg_index i ON i.indexrelid = s.indexrelid
JOIN pg_class c ON c.oid = s.indexrelid
JOIN pg_am am ON am.oid = c.relam
ORDER BY pg_relation_size(s.indexrelid) DESC;
```

19. Interpretation Guidelines

- Treat statistics as evidence over an observation window, not as permanent truth.
- `idx_scan=0` means no counted scans in the retained statistics period; it does not prove the index is unnecessary.
- Primary-key and unique indexes can enforce data integrity even when query scan counts are low.
- High leaf fragmentation is a signal to investigate, not a universal REINDEX trigger.
- Low average leaf density can be intentional or workload-driven; compare with fillfactor, update/delete patterns and historical baselines.
- Measure query performance before and after maintenance when the objective is performance improvement.
- Account for WAL generation, replica replay, storage I/O and temporary disk needs during rebuilds.
- Prefer concurrent rebuilds for busy production tables when its restrictions and extra cost are acceptable.

20. Real-World Production Scenarios

20.1 Scenario - Large Index with Zero Scans

Symptom: a 40 GB index reports `idx_scan = 0`.

What to check: statistics reset history, primary/unique status, `pg_constraint` dependencies, query plans, batch/month-end workload, application releases and replica usage.

Corrective action: only after a representative observation window and dependency review should `DROP INDEX CONCURRENTLY` be considered. This SOP intentionally does not auto-generate `DROP` commands.

20.2 Scenario - B-tree Growth After Heavy Deletes

Symptom: the table shrank logically after retention deletes, but the index remains large.

What to check: `pgstatindex()` density/fragmentation, index size trend, workload, free disk and maintenance window.

Corrective action: if evidence supports inefficient index space use, plan `REINDEX`. For a busy production table, evaluate `REINDEX CONCURRENTLY`.

20.3 Scenario - Concurrent Reindex Fails

Symptom: `REINDEX CONCURRENTLY` fails and leaves an `INVALID` transient index.

What to check: `psql \d` output and the transient index suffix. PostgreSQL documentation describes `_ccnew` and `_ccold` recovery cases. Follow the documented recovery procedure rather than dropping arbitrary indexes.

21. Troubleshooting

Symptom	Likely cause / check	Action
<code>pgstatindex()</code> does not exist	<code>pgstattuple</code> extension is not installed in this database.	Install with <code>CREATE EXTENSION pgstattuple</code> if approved.
permission denied for <code>pgstatindex</code>	Caller lacks required function privilege.	Use an authorized role; consider <code>pg_stat_scan_tables</code> membership.
<code>pgstatindex</code> says wrong index type / function fails	Target is not a physical B-tree index.	Check <code>pg_am</code> and <code>relkind</code> first.
<code>idx_scan</code> unexpectedly zero	Statistics may have been reset or workload not observed.	Check observation window and stats reset context before action.
<code>REINDEX</code> blocks writers	Standard <code>REINDEX</code> takes stronger locking.	Evaluate <code>REINDEX CONCURRENTLY</code> and its caveats.
Concurrent rebuild takes much longer	Expected: extra scans/work and waits for transactions.	Monitor progress and system load; do not assume it is hung.

22. Security and Least Privilege

- Do not grant superuser solely for index reporting.
- Use the minimum statistics/catalog privileges required by the operational role.
- `pgstattuple` page-level inspection is restricted by default; use controlled role membership or explicit grants according to policy.
- Keep maintenance execution roles separate from read-only monitoring roles where practical.
- Audit `REINDEX` and `DROP INDEX` changes through the normal change-management process.

23. Performance and Operational Impact

Catalog and `pg_stat_user_indexes` queries are lightweight compared with page-level inspection. `pgstatindex()` scans index pages to derive its metrics, so broad database-wide use can generate I/O. Filter large reports by schema, index size or a candidate list.

REINDEX is a maintenance operation with CPU, I/O, WAL and storage implications. REINDEX CONCURRENTLY is designed to allow normal writes to continue, but PostgreSQL documents that it performs more total work and generally takes longer.

24. Production Readiness Checklist

- Representative statistics window captured.
- Critical/rare workloads reviewed.
- Primary, unique and constraint indexes identified.
- Candidate indexes validated with query plans.
- `pgstattuple` access follows least privilege.
- Large `pgstatindex` scans scheduled safely.
- Disk headroom checked before REINDEX.
- WAL/replication impact considered.
- Maintenance command peer-reviewed.
- REINDEX CONCURRENTLY restrictions reviewed.
- Post-change index validity and query performance validated.
- Rollback/recovery approach documented.

25. Post-Implementation Validation

```
-- Confirm invalid user indexes:
SELECT
    n.nspname AS schema_name,
    c.relname AS index_name,
    i.indisvalid,
    i.indisready
FROM pg_index i
JOIN pg_class c ON c.oid = i.indexrelid
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname NOT IN ('pg_catalog', 'information_schema')
    AND (NOT i.indisvalid OR NOT i.indisready)
ORDER BY 1,2;

-- Re-check size and usage:
SELECT
    schemaname, relname, indexrelname,
    idx_scan, last_idx_scan,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
ORDER BY pg_relation_size(indexrelid) DESC;
```

26. Operational Cheat Sheet

Task	Command / View
List index definitions	pg_indexes
Usage statistics	pg_stat_user_indexes
Access method	pg_class.relam -> pg_am
Index metadata	pg_index
Index bytes	pg_relation_size(indexrelid)
Pretty size	pg_size_pretty(...)
B-tree page health	pgstatindex(regclass)
Rebuild one index	REINDEX INDEX ...
Reduced write blocking	REINDEX INDEX CONCURRENTLY ...
Progress	pg_stat_progress_create_index

27. Key Takeaways

- Index administration is a measurement-and-validation process, not a threshold-only process.
- Use pg_stat_user_indexes for usage, catalogs for metadata, size functions for footprint, and pgstatindex() for B-tree page-level diagnostics.
- Do not equate zero scans with safe-to-drop.
- Do not call fragmentation or free space an exact universal bloat percentage.
- REINDEX is appropriate for specific conditions such as corruption, inefficient B-tree space use, or applying changed storage parameters.
- For live production systems, evaluate REINDEX CONCURRENTLY when keeping writes available is important.

28. Official References

PostgreSQL 18 - Chapter 11: Indexes: <https://www.postgresql.org/docs/18/indexes.html>

PostgreSQL 18 - pg_indexes: <https://www.postgresql.org/docs/18/view-pg-indexes.html>

PostgreSQL 18 - Cumulative Statistics System: <https://www.postgresql.org/docs/18/monitoring-stats.html>

PostgreSQL 18 - pgstattuple / pgstatindex: <https://www.postgresql.org/docs/18/pgstattuple.html>

PostgreSQL 18 - REINDEX: <https://www.postgresql.org/docs/18/sql-reindex.html>

PostgreSQL 18 - Routine Reindexing: <https://www.postgresql.org/docs/18/routine-reindex.html>

PostgreSQL 18 - Monitoring Database Activity: <https://www.postgresql.org/docs/18/monitoring.html>

Appendix A - Complete Candidate Review Script

```
-- 1) Inventory and usage
SELECT
    s.schemaname,
    s.relname AS table_name,
    s.indexrelname AS index_name,
    am.amname AS access_method,
    i.indisprimary,
    i.indisunique,
    i.indisvalid,
    s.idx_scan,
```

```

    s.last_idx_scan,
    pg_relation_size(s.indexrelid) AS index_bytes,
    pg_size_pretty(pg_relation_size(s.indexrelid)) AS index_size
FROM pg_stat_user_indexes s
JOIN pg_index i ON i.indexrelid = s.indexrelid
JOIN pg_class c ON c.oid = s.indexrelid
JOIN pg_am am ON am.oid = c.relam
ORDER BY pg_relation_size(s.indexrelid) DESC;

```

-- 2) Zero-scan review candidates, excluding primary/unique

```

SELECT
    s.schemaname,
    s.relname,
    s.indexrelname,
    pg_size_pretty(pg_relation_size(s.indexrelid)) AS index_size,
    s.idx_scan,
    s.last_idx_scan
FROM pg_stat_user_indexes s
JOIN pg_index i ON i.indexrelid = s.indexrelid
WHERE s.idx_scan = 0
    AND NOT i.indisprimary
    AND NOT i.indisunique
ORDER BY pg_relation_size(s.indexrelid) DESC;

```

-- 3) Physical B-tree candidates >= 100 MB

```

SELECT
    n.nspname,
    t.relname AS table_name,
    c.relname AS index_name,
    c.oid::regclass AS index_regclass,
    pg_size_pretty(pg_relation_size(c.oid)) AS index_size
FROM pg_class c
JOIN pg_index i ON i.indexrelid = c.oid
JOIN pg_class t ON t.oid = i.indrelid
JOIN pg_namespace n ON n.oid = c.relnamespace
JOIN pg_am am ON am.oid = c.relam
WHERE c.relkind = 'i'
    AND am.amname = 'btree'
    AND n.nspname NOT IN ('pg_catalog', 'information_schema')
    AND pg_relation_size(c.oid) >= 100 * 1024 * 1024
ORDER BY pg_relation_size(c.oid) DESC;

```

Appendix B - DBA Decision Matrix

Signal	Confidence	Automatic action?	DBA response
idx_scan = 0	Low by itself	No	Validate observation window, constraints and rare workloads.
Large index	Low by itself	No	Trend growth and correlate with workload.
Low leaf density	Medium diagnostic signal	No	Check fillfactor, churn and historical baseline.
High leaf fragmentation	Medium diagnostic signal	No	Correlate with performance/size and maintenance goals.
Known corruption	High	No automatic action	Plan supported REINDEX recovery.
Persistent inefficient B-tree space use	High after validation	No automatic action	Plan REINDEX; evaluate CONCURRENTLY in production.